Technology Stack Document

Arati Business ERP

Document Version: 1.1
Status: Proposed / Security-Aligned Technology Baseline
Project: Arati Business ERP
Primary Architecture: Modular Monolith
Primary Database: PostgreSQL
Accounting Integration: TallyPrime
Related Documents: `BRD.md`, `SRS.md`, `ARCHITECTURE.md`, `DATABASE.md`, `API.md`, `PROTECTION.md`

---

1. Purpose

This document defines the proposed technologies, frameworks, libraries, infrastructure, development tools, and engineering standards for the Arati Business ERP.

The stack must support a real production ERP, not merely a prototype.

The technology baseline is designed around:

- correctness of business transactions
- inventory integrity
- financial integrity
- strong security
- auditability
- maintainability
- operational simplicity
- reasonable infrastructure cost
- future scalability
- resilient TallyPrime integration

The newly added `PROTECTION.md` is now a direct input into technology selection.

---

2. Technology Principles

2.1 Mature Technologies

Core infrastructure should use mature and actively maintained technologies.

2.2 Type Safety

TypeScript should be the primary application language wherever practical.

2.3 Secure by Default

Security controls must be built into the architecture rather than added after the application is complete.

2.4 Simple Before Complex

Do not introduce infrastructure merely because it is popular.

Every additional service creates:

- operational responsibility
- failure modes
- security surface
- backup requirements
- upgrade requirements
- monitoring requirements

2.5 Production First

The stack must support:

- transactions
- concurrency
- backups
- monitoring
- secure deployments
- controlled migrations
- auditability

2.6 Modular Architecture

The codebase remains a modular monolith initially.

The application should have strong internal module boundaries so that future extraction is possible if genuinely required.

2.7 Open Standards

Prefer:

- HTTPS
- REST
- OpenAPI
- SQL
- JSON
- standard authentication/security concepts

2.8 PostgreSQL Is Authoritative

PostgreSQL is the authoritative transactional source for ERP data.

Redis, browser state, caches, queues, and TallyPrime must never become the source of truth for ERP inventory or financial state.

---

3. Proposed Stack Summary

Layer	Technology	Status	Reason
Language	TypeScript	Proposed	One strongly typed language
Frontend	Next.js + React	Proposed	Mature web application platform
Styling	Tailwind CSS	Proposed	Consistent UI system
Backend	NestJS	Proposed	Structured modular backend
API	REST + OpenAPI	Proposed	Clear integration contract
Database	PostgreSQL	Proposed	Strong transactions and relational integrity
ORM	Prisma	Proposed	Type-safe data access and migrations
Validation	Zod and/or Nest-compatible validation	To finalize	Runtime input validation

Authentication	Secure server-managed session architecture	Preferred	Strong browser security
Password Hashing	Argon2id or equivalent approved adaptive hash	Preferred	Strong password protection
Authorization	RBAC + granular permissions	Approved concept	Least privilege
Cache	PostgreSQL first; Redis only if justified	Conditional	Avoid unnecessary infrastructure
Queue	BullMQ + Redis only if justified	Conditional	Reliable async work when actually required
Rate Limiting	Application/infrastructure rate limiting; Redis if needed	Conditional	Security without mandatory Redis dependency
File Storage	S3-compatible object storage	Proposed	Separate binary storage from DB
Testing	Vitest + Playwright	Proposed	Unit + E2E
API Testing	Supertest / equivalent	Proposed	Backend integration testing
Documentation	Markdown + OpenAPI	Proposed	Engineering and API contracts
Version Control	Git	Proposed	Standard source control
Repository	GitHub	Proposed	Collaboration and CI/CD
CI/CD	GitHub Actions	Proposed	Automated quality/security checks
Containers	Docker	Proposed	Reproducible deployment
Reverse Proxy	Nginx / managed ingress	To finalize	TLS, routing, edge controls
Observability	OpenTelemetry-compatible approach	Proposed	Tracing/metrics/log correlation
Error Tracking	Sentry / equivalent	To finalize	Application error visibility

Secrets	Environment secrets / managed secrets service	Required	Prevent credential leakage
Security Scanning	Dependency + secret + container scanning	Required	Supply-chain protection
Deployment	Secure VPS/cloud/container deployment	To finalize	Cost-conscious production
Accounting	TallyPrime	Business requirement	External accounting system

---

4. Important Stack Change: Redis Is Not Mandatory

The previous technology baseline treated Redis and BullMQ as generally proposed infrastructure.

That is now revised.

Redis should **not** be installed simply because the ERP might eventually need queues or caching.

The initial system should work correctly without Redis for core ERP transactions.

Redis may be introduced when one or more of these are genuinely required:

- reliable background job processing
- distributed rate limiting
- short-lived distributed cache
- multi-instance coordination
- workload that should not run inside API processes

If the first production deployment does not need these capabilities, Redis should be omitted.

This reduces:

- cost
- operational complexity
- attack surface
- failure modes
- backup/monitoring requirements

---

5. Frontend

5.1 Next.js

Role: Web application framework.

Next.js is proposed for:

- application routing
- authenticated ERP screens
- server/client rendering where appropriate
- frontend structure
- API consumption
- performance optimization

The ERP is primarily an authenticated business application.

Public marketing/website functionality should not dictate the architecture of the ERP application.

---

6. React

React provides the component model.

The UI should use reusable components.

Examples:

Button
Input
Select
DatePicker
DataTable
Modal
Drawer
Form
StatusBadge
Pagination
SearchBox
MoneyInput
QuantityInput
SerialSelector
BatchSelector

Security-sensitive UI controls must never be treated as the actual security boundary.

For example, hiding an "Adjust Stock" button does not authorize or protect the operation.

---

7. TypeScript

TypeScript shall be used across:

- frontend
- backend
- shared contracts where useful
- utilities
- API schemas
- integration adapters

Strict TypeScript configuration should be enabled.

Avoid `any` unless there is a documented reason.

---

8. UI and Styling

8.1 Tailwind CSS

Tailwind CSS remains proposed.

The project should define a consistent design system rather than allowing arbitrary styling throughout the application.

8.2 Component Library

A component library may be adopted after the UX requirements are clearer.

Do not introduce a large UI framework solely for convenience.

The final component library remains **TBD — Technical Decision Required**.

---

9. Backend

9.1 NestJS

NestJS is proposed for the backend.

The backend should be divided into domain modules such as:

auth
users
organization
parties
products
pricing
purchases
inventory
sales
payments
returns
fulfilment
warranty
reports
notifications
audit
tally

The modules should communicate through defined application/domain interfaces rather than uncontrolled cross-module database access.

---

10. Backend Layering

Recommended structure:

HTTP / Controller
↓
Application Service
↓
Domain Rules
↓
Repository / Persistence
↓
PostgreSQL

External systems:

Domain/Application
↓

Integration Interface
↓
Infrastructure Adapter
↓
External System

Example:

Tally Module
↓
Tally Interface
↓
Tally Adapter
↓
TallyPrime

This prevents Tally-specific implementation details from leaking throughout the ERP.

---

11. API

11.1 REST

The application API will use REST.

Base path:

/api/v1

Examples:

GET /api/v1/products
POST /api/v1/products
GET /api/v1/products/:id
PATCH /api/v1/products/:id

GET /api/v1/sales-invoices
POST /api/v1/sales-invoices
POST /api/v1/sales-invoices/:id/post

API details are maintained in `API.md`.

---

12. OpenAPI

OpenAPI shall describe:

- endpoints
- request schemas
- response schemas
- authentication
- authorization expectations
- error responses
- enums
- examples

OpenAPI should become part of the development workflow rather than a document written only at the end.

---

13. API Security Technology

The API layer should include:

- authentication middleware/guards
- authorization guards
- schema validation
- request-size limits
- rate limiting where appropriate
- safe pagination
- safe filtering/sorting
- consistent error handling
- request/correlation IDs
- security headers at the appropriate layer

The backend remains the security boundary.

---

14. Authentication

14.1 Preferred Browser Model

For the ERP web application, prefer a secure server-managed session architecture using:

- HTTPS

- Secure cookies
- HttpOnly cookies
- appropriate SameSite settings
- session expiration
- session revocation
- server-side session validation

The exact session implementation remains **TBD — Security Implementation Decision**.

Avoid long-lived authentication secrets in browser-accessible storage.

14.2 Password Hashing

Use a modern adaptive password hashing algorithm.

Preferred baseline:

Argon2id

An equivalent approved algorithm may be selected if implementation constraints require it.

Plaintext passwords are prohibited.

---

15. Authentication Security

The authentication system must support:

Login
Logout
Session expiration
Session revocation
Password reset
Account deactivation
Failed-login protection
Privileged-account protection

MFA should be supported for privileged users when introduced.

Exact MFA scope is **TBD — Security Decision Required**.

---

16. Authorization

Authorization uses:

User
↓
Role
↓
Permission
↓
Action

Example:

inventory.view
inventory.adjust
inventory.transfer

sales.invoice.create
sales.invoice.post
sales.invoice.cancel

payments.create
payments.post

tally.sync
tally.reconcile

Authorization must be enforced on the backend.

---

17. Object-Level Authorization

The technology architecture must support authorization at the resource level.

For example:

GET /sales-invoices/{id}

must verify:

authenticated user
↓
permission
↓
organization
↓
branch
↓

resource access

Changing an ID in a URL must never allow access to another branch's data.

---

18. Organization and Branch Security

The stack must support application-level tenant/scope checks for:

- organization
- branch
- warehouse
- other restricted resources

The backend should derive access from authenticated context rather than trusting client-supplied IDs.

---

19. Database

19.1 PostgreSQL

PostgreSQL is the authoritative transactional database.

It must support:

- ACID transactions
- foreign keys
- unique constraints
- check constraints
- row-level locking
- reliable migrations
- indexed queries
- reporting queries
- inventory movement history
- financial references

PostgreSQL is the system of record.

---

20. ORM

20.1 Prisma

Prisma remains the proposed ORM.

Use it for:

- normal CRUD
- relationships
- migrations
- type-safe queries

However:

Correctness takes priority over ORM purity.

Carefully reviewed SQL may be used for:

- complex reporting
- locking/concurrency operations
- performance-critical inventory queries
- database-specific constraints
- advanced PostgreSQL features

---

21. Database Security

The technology stack must support:

- private database networking
- separate database roles where practical
- encrypted connections where appropriate
- least-privilege credentials
- encrypted backups
- controlled migrations
- monitoring

The application database must not be publicly exposed.

---

22. Database Migration Technology

All schema changes must use version-controlled migrations.

Process:

Schema Change
↓
Migration
↓
Code Review
↓
Automated Tests
↓
Staging
↓
Production

Manual production schema edits are prohibited except under a documented emergency procedure.

---

23. Validation

Validation should occur at multiple layers:

Frontend Validation
↓
API Schema Validation
↓
Domain Validation
↓
Database Constraints

Frontend validation is for usability.

Backend/domain validation is for correctness and security.

Database constraints provide an additional integrity boundary.

---

24. Validation Technology

A runtime validation library is required.

Candidates:

- Zod
- NestJS-compatible validation mechanisms
- another reviewed schema-validation library

The final choice should avoid maintaining two conflicting schema definitions unnecessarily.

TBD — Technical Decision Required.

---

25. Inventory Technology Requirements

The technology must support:

- atomic stock operations
- transaction boundaries
- row/database locking
- serial uniqueness
- batch tracking
- immutable/append-oriented movement history
- balance projection
- transaction rollback
- concurrency protection

Architecture:

Purchase
↓
Inventory Engine
↑
Sales
↑
Returns / Transfers / Adjustments

Inventory balances must never be treated as a freely editable cache.

---

26. Inventory Concurrency

The stack must support scenarios such as:

Stock = 1

Request A → sell 1
Request B → sell 1

Only a valid transaction should succeed.

Technology choices must therefore support:

- PostgreSQL transactions
- row locking where necessary
- atomic updates
- unique constraints
- appropriate isolation

Exact implementation belongs in the database/domain design.

---

27. Financial Integrity

Technology must support safe handling of:

- money
- tax
- discounts
- payments
- allocations
- credit
- cancellations
- returns

Money values should use PostgreSQL numeric/decimal representations and application decimal-safe handling.

Floating-point arithmetic must not be used for authoritative financial amounts.

---

28. Redis

Redis is **conditional**, not mandatory.

Potential uses:

short-lived cache
distributed rate limiting
background queues
temporary coordination

Redis must never be the authoritative source for:

inventory
invoice totals
payments

receivables
financial state

If Redis is introduced, it must have:

- authentication
- private networking
- controlled access
- monitoring
- failure handling

---

29. Background Jobs

Background processing is appropriate for:

- Tally synchronization
- large report generation
- exports
- notifications
- file processing
- maintenance tasks

Recommended technology if job requirements justify it:

BullMQ
↓
Redis

However, the project should not introduce BullMQ/Redis before a real asynchronous workload exists.

Core ERP transactions must not depend on a queue being available.

---

30. TallyPrime Integration

TallyPrime remains an external accounting system.

Architecture:

ERP Transaction
↓
ERP Database Commit

↓
Integration Job / Outbox-style Record
↓
Tally Adapter
↓
TallyPrime

The exact mechanism is **TBD — Technical Verification Required**.

The stack must support:

- retries
- idempotency
- failure states
- mapping
- reconciliation
- auditability

Tally failure must not roll back a valid ERP operational transaction merely because synchronization is unavailable.

---

31. Tally Security

Tally credentials/configuration must:

- remain server-side
- never reach browser code
- never appear in ordinary API responses
- never be logged
- be stored in protected configuration/secrets
- be rotatable

Tally synchronization should operate through a dedicated integration boundary.

---

32. File Storage

Use S3-compatible object storage for:

- product images
- purchase documents
- invoices/attachments
- warranty documents

- exports

PostgreSQL should store metadata and references rather than large binary files by default.

---

33. File Security

File upload infrastructure must support:

- maximum file size
- allowed file types
- file-signature validation where practical
- safe filenames
- private storage
- authorization before download
- malware scanning where practical
- no executable upload paths
- path traversal protection

Browser MIME type alone must not be trusted.

---

34. Caching

Caching should be conservative.

Good candidates:

static configuration
read-mostly master data
short-lived report results

Bad candidates:

authoritative inventory
financial balances
payment state
critical transaction state

Stale cache data must never be allowed to make a financial or stock transaction incorrect.

---

35. Search

PostgreSQL should handle initial search.

Examples:

- product name
- SKU
- barcode
- model
- serial number
- party name
- invoice number

A dedicated search engine should only be introduced if actual data volume or performance requires it.

---

36. Reporting

Initial reports should query PostgreSQL.

For expensive reports:

User
↓
Report Request
↓
Background Job
↓
Report Generation
↓
Stored Result
↓
Secure Download

A separate analytics database is not required for MVP.

---

37. Logging

Use structured application logs.

Logs should include appropriate:

- timestamp
- severity
- service/module
- request ID
- operation
- error classification

Never log:

passwords
access tokens
session secrets
Tally credentials
encryption keys
unnecessary sensitive personal data

---

38. Observability

The application should use an OpenTelemetry-compatible approach where practical.

Important telemetry:

request
↓
controller
↓
application service
↓
database
↓
background job
↓
external integration

Track:

- availability
- latency
- error rates
- database health
- job health
- Tally synchronization failures
- storage failures

---

39. Error Tracking

A dedicated error-tracking service such as Sentry may be used.

The final provider must be selected based on:

- privacy
- cost
- data residency
- integration quality
- operational requirements

Error tracking must not capture secrets or unnecessary sensitive data.

---

40. Security Tooling

Security checks should include:

Dependency vulnerability scanning
Secret scanning
Static analysis where useful
Container image scanning
Authentication tests
Authorization tests
Input validation tests
API security tests

The CI pipeline should fail on appropriately severe security findings.

Exact thresholds are **TBD — Engineering/Security Decision Required**.

---

41. Dependency Management

Dependencies must be:

- locked with a package lockfile
- regularly reviewed
- vulnerability-scanned

- removed when unnecessary
- updated when security fixes are required

Avoid unnecessary dependency count.

Each dependency adds supply-chain risk.

---

42. Container Security

Docker is proposed for reproducible deployment.

Requirements:

- minimal trusted base images
- pinned versions where practical
- image vulnerability scanning
- non-root processes where practical
- minimal Linux capabilities
- secrets supplied at runtime
- no production secrets baked into images
- regular image rebuilding

Potential containers:

web
api
worker (if needed)

Database and Redis containers may be used in development.

Production should consider managed PostgreSQL/Redis where operationally justified.

---

43. Deployment

The initial production deployment should remain simple.

Possible architecture:

Internet
↓
DNS / Edge / WAF
↓
Reverse Proxy / Ingress

↓
Web + API
↓
PostgreSQL
↓
Object Storage

Optional only when needed:
↓
Worker
↓
Redis
↓
TallyPrime

Redis and worker infrastructure are optional until required.

---

44. Outer Security Technology

The production environment should support:

- HTTPS/TLS
- firewall rules
- private database networking
- reverse proxy/ingress
- WAF where appropriate
- rate limiting
- request-size limits
- server hardening
- secure administrative access
- monitoring

The exact infrastructure provider remains **TBD — Deployment Decision Required**.

---

45. Web Security

The frontend/deployment stack should support:

- HTTPS
- secure cookies
- CSRF protection where cookie authentication requires it
- CORS restrictions

- Content Security Policy
- HSTS
- X-Content-Type-Options
- Referrer-Policy
- frame protection
- safe output encoding

Security headers should be implemented at the appropriate application/edge layer.

---

46. CORS

Authenticated ERP APIs must use an explicit CORS allowlist.

Do not use unrestricted:

Access-Control-Allow-Origin: *

for private authenticated APIs.

Allowed origins should be environment-specific.

---

47. CSRF

If cookie-based authentication is used, CSRF protection must be implemented using an appropriate combination of:

- SameSite cookie controls
- CSRF tokens where required
- origin validation where appropriate

Exact implementation depends on the final authentication framework.

---

48. Rate Limiting

Rate limiting is required for high-risk endpoints such as:

login
password reset
token/session operations

file uploads
public endpoints

Rate limiting may initially be implemented at the application/edge layer.

Redis should only be introduced if distributed rate limiting is required.

---

49. Secrets Management

Secrets must never be committed to Git.

Examples:

DATABASE_URL
AUTH_SECRET
TALLY credentials
STORAGE credentials
API keys
encryption keys
SMTP credentials

Preferred deployment model:

Application
↓
Environment / Managed Secrets
↓
Runtime

For larger deployments, a dedicated secrets-management service may be used.

---

50. Environment Separation

At minimum:

Development
Staging
Production

Each should have separate:

- database
- credentials

- storage
- API keys
- Tally configuration
- authentication secrets

Production data should not be copied into development without appropriate anonymization.

---

51. CI/CD

GitHub Actions is proposed.

Pipeline:

Pull Request
↓
Install
↓
Lint
↓
Type Check
↓
Unit Tests
↓
Security Scans
↓
Build
↓
Integration Tests
↓
Container Scan
↓
Staging
↓
Approval
↓
Production

Production deployment should have appropriate approval controls.

---

52. Repository Protection

GitHub repository protection should include:

- protected main branch
- pull-request review
- required CI checks
- secret scanning
- dependency/security checks
- restricted production deployment
- controlled repository permissions

No production secrets belong in Git.

---

53. Testing Stack

Unit

Vitest

Focus heavily on:

- pricing
- taxes
- inventory
- payments
- credit rules
- document state transitions
- permissions
- Tally mapping

Integration

Test:

- PostgreSQL operations
- transactions
- inventory concurrency
- sales
- purchases
- payments
- returns
- authorization
- Tally boundaries

End-to-End

Playwright

Critical E2E path:

Login
↓
Create Product
↓
Receive Purchase
↓
Verify Stock
↓
Create Sale
↓
Receive Payment
↓
Verify Outstanding
↓
Verify Inventory
↓
Verify Tally Queue

---

54. Security Testing

Security tests must verify that:

Frontend bypass
↓
API
↓
Backend authorization
↓
Domain rules
↓
Database

cannot be bypassed.

Important tests:

- unauthorized branch access
- unauthorized warehouse access
- ID manipulation
- privilege escalation
- price override abuse
- stock adjustment abuse
- duplicate posting
- payment manipulation

- session abuse
- file upload abuse
- Tally credential exposure

Penetration testing should be considered before production launch.

---

55. Database Backup Technology

Backups must support:

- automated scheduling
- encryption
- restricted access
- retention
- restoration testing
- separation from primary infrastructure

The final backup provider and retention schedule are **TBD — Operations Decision Required**.

---

56. Disaster Recovery

The technology stack must support recovery from:

- database failure
- server failure
- deployment failure
- storage failure
- accidental deletion
- destructive attack
- Tally outage

RPO and RTO are:

TBD — Business/Operations Decision Required.

---

57. Local Development

Target developer experience:

Clone repository
↓
Install dependencies
↓
Configure environment
↓
Start required infrastructure
↓
Run migrations
↓
Seed development data
↓
Start application

Development infrastructure should not require unnecessary cloud services.

A local PostgreSQL instance/container should be sufficient for normal development.

Redis should only be required locally when working on features that actually depend on it.

---

58. Development Seed Data

Development/test environments may contain:

Demo Organization
Demo Branch
Demo Warehouse
Demo Products
Demo Parties
Demo Users
Demo Price Lists
Demo Transactions

Seed scripts must never overwrite production data.

---

59. Frontend State Management

Do not introduce a global state-management library without a demonstrated need.

Preferred:

Local state
+
Server/API state
+
URL state

Global client state should be limited to genuinely global UI/application concerns.

Exact server-state library remains **TBD — Frontend Decision Required**.

---

60. Form Management

Forms must support:

- schema validation
- dirty state
- submission state
- clear validation errors
- server-side errors
- optimistic concurrency feedback where applicable

The final form library is **TBD — Technical Decision Required**.

---

61. UI Performance

Optimize operationally important workflows:

- POS
- product search
- large tables
- inventory lookup
- party lookup
- invoice entry

Use:

- pagination
- debounced search
- virtualization
- efficient queries
- caching only where justified

Do not sacrifice correctness for UI speed.

---

62. Export Generation

Small exports may be synchronous.

Large exports should be asynchronous:

Request
↓
Job
↓
Generate
↓
Private Storage
↓
Expiring Download

Export authorization must be checked before generating or downloading sensitive business data.

---

63. Technology Decisions Not Yet Frozen

The following remain open:

- exact Next.js version
- exact NestJS version
- exact PostgreSQL version
- exact Prisma version
- authentication/session implementation
- validation library
- UI component library
- form library
- server-state library
- error tracking provider
- hosting provider
- object storage provider
- reverse proxy/edge provider
- Tally integration mechanism
- monitoring provider
- secrets-management provider
- whether Redis is required

- whether BullMQ is required

These should be finalized based on implementation requirements rather than preference alone.

---

64. Technology Decisions That Are Now Stronger

The following principles should be treated as architectural constraints:

Required

- TypeScript
- PostgreSQL
- secure backend authorization
- database transactions
- server-side validation
- auditability
- secure secrets handling
- HTTPS in production
- protected database networking
- dependency/security scanning
- automated testing
- version-controlled migrations

Preferred

- Next.js
- React
- NestJS
- Prisma
- REST/OpenAPI
- Docker
- Playwright
- Vitest
- OpenTelemetry-compatible observability
- Argon2id
- secure server-managed browser sessions

Conditional

- Redis
- BullMQ

- Sentry
- WAF
- managed Redis
- dedicated search engine
- analytics database

---

65. Cost Strategy

The MVP should minimize recurring infrastructure cost without compromising security or reliability.

Priority:

1. Do not pay for infrastructure we do not need.
2. Do not remove security controls merely to save money.
3. Prefer managed services when they materially reduce operational risk.
4. Start with PostgreSQL as the primary data platform.
5. Add Redis only when a real requirement appears.
6. Add specialized infrastructure only when PostgreSQL/application architecture is demonstrably insufficient.

---

66. Technology Change Policy

Technology should only change when there is a clear reason.

Evaluate:

- business impact
- security
- migration cost
- performance
- maintenance
- developer productivity
- operational complexity
- vendor risk
- long-term support

Architecture-affecting changes should be documented through an Architecture Decision Record.

---

67. Relationship With Other Documents

BRD.md
Business requirements
↓
SRS.md
Software requirements
↓
ARCHITECTURE.md
System structure
↓
PROTECTION.md
Security / protection requirements
↓
TECHSTACK.md
Technology implementation choices
↓
DATABASE.md
Data model
↓
API.md
API contracts
↓
ERD.md
Relationship validation
↓
TALLY-INTEGRATION.md
Accounting integration
↓
DESIGN.md
UI/UX and product design
↓
PHASES.md
Delivery roadmap

---

68. Why This Stack

The proposed stack intentionally stays conventional.

It provides:

- one primary language
- strong typing
- mature relational transactions
- strong database integrity

- modular backend structure
- modern web UI
- secure application boundaries
- automated testing
- CI/CD
- containerized deployment
- clear integration boundaries
- a path to scale without premature microservices

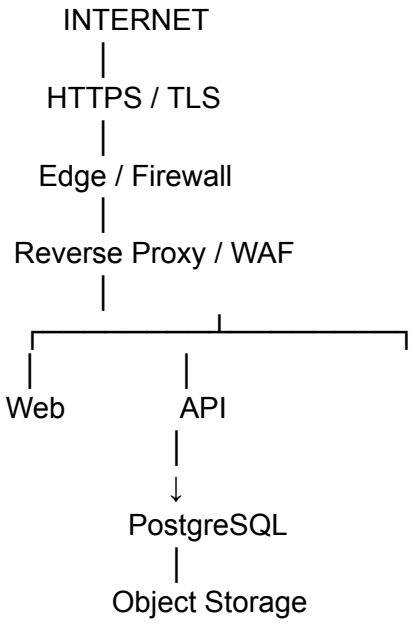
The most important change is that infrastructure is now **requirement-driven**.

We will not add Redis, a search engine, an analytics database, microservices, or other infrastructure simply because the application may someday need them.

---

69. Recommended Initial Production Shape

The preferred first deployment is approximately:



Optional only when needed:



TallyPrime

This keeps the core system simple while preserving a path to background processing.

---

70. Definition of Technology Baseline Done

The technology baseline can be promoted from proposed to approved when:

- security architecture is reviewed
- authentication implementation is selected
- deployment architecture is selected
- PostgreSQL version is selected
- Node/TypeScript framework versions are selected
- validation approach is selected
- API contract is reviewed
- ERD/database design is reviewed
- Tally communication mechanism is verified
- backup/recovery approach is tested
- production observability is selected
- CI/CD security checks are operational
- critical business workflows pass automated tests

---

71. Next Technology Decisions

Before implementation becomes production-grade, the next decisions should be:

1. Finalize `ERD.md`
2. Finalize business workflows
3. Select authentication/session implementation
4. Select validation approach
5. Verify TallyPrime integration method
6. Decide initial deployment model
7. Decide whether Redis/BullMQ is actually required for Phase 1
8. Define backup/recovery architecture
9. Define security testing requirements
10. Freeze the initial package/version baseline

Document Status

Version: 1.1
Status: Proposed / Security-Aligned Technology Baseline
Major update: Integrated `PROTECTION.md` requirements and changed Redis/BullMQ from default infrastructure to conditional infrastructure.